TERRAFORM ON AWS

Provisioning an EC2 Web Server with a VPC, using Infrastructure as Code

DW **Terraform AWS Project**

Built with Ubuntu Linux, Visual Studio Code, and the Terraform AWS provider

Introducing This Project

In this project, I used Terraform to provision a small piece of AWS infrastructure entirely as code. The configuration builds a custom VPC, a public subnet, an internet gateway and route table, a security group, and an EC2 instance running Amazon Linux 2. I wrote the Terraform files using Visual Studio Code on Ubuntu Linux, then used the Terraform CLI to initialize, plan, and apply the configuration against my AWS account.

Once the EC2 instance launches, Terraform connects to it over SSH and runs a provisioner that copies a shell script to the instance and executes it. That script installs and starts Apache (httpd), publishes a simple HTML page, and then installs Docker so it can run an Nginx container exposed on port 8080. The result is a single command, terraform apply, that takes a brand-new AWS account from nothing to a running web server and a running container.

Tools and Concepts

I used Terraform, the AWS provider for Terraform, the AWS CLI/console for verification, Visual Studio Code as my editor, SSH key pairs for secure access, and shell scripting to configure the instance after launch. Key concepts include declarative infrastructure as code, Terraform's resource and data source model, input variables and tfvars files for separating configuration from code, output values for surfacing useful information after a deployment, and provisioners for running remote commands as part of a resource's lifecycle.

Working through this project reinforced how a VPC, subnet, internet gateway, and route table fit together to give an EC2 instance a path to the public internet, and how security groups act as a virtual firewall controlling exactly which ports are reachable.

Project Reflection

I chose to build this project to get hands-on practice with Terraform's core workflow, writing reusable variables, wiring up networking from scratch, and using provisioners to bootstrap an instance automatically instead of configuring it by hand after launch.

Project Overview

This diagram illustrates the high-level workflow for provisioning AWS networking and compute resources with Terraform, then bootstrapping the EC2 instance with a web server and a containerized app.

PART 1

Write the Terraform configuration

- main.tf defines the provider, VPC, subnet, gateway, route table, security group, key pair, AMI lookup, and EC2 instance
- variables.tf declares input variables with no hard-coded values inside main.tf
- terraform.tfvars supplies the actual values for each variable

PART 2

Provision networking and compute

- terraform init downloads the AWS provider plugin
- terraform plan shows the resources that will be created
- terraform apply creates the VPC, subnet, internet gateway, route table, security group, key pair, and EC2 instance in us-east-2

PART 3

Bootstrap the instance

- A file provisioner copies entry-script.sh to the new EC2 instance over SSH
- A remote-exec provisioner runs the script, which installs Apache and serves a test page
- The script also installs Docker and runs an Nginx container on port 8080
- outputs.tf prints the instance's public IP, the VPC ID, and the AMI ID after apply completes

Project Set Up

Terraform Provider, Remote State, and VPC

My main.tf begins with a required_providers block that pins the AWS provider to a specific version. Right alongside it, I configured Terraform to use an Amazon S3 backend for remote state storage with encryption and state locking to support collaborative infrastructure management and reduce the risk of state conflicts. From there, the provider “aws” block sets the region to us-east-2, and I defined an aws_vpc resource named “main” using a CIDR block supplied through a variable, with a Name tag that combines a static string with another variable so the VPC's name is easy to identify in the AWS console.

The provider block no longer includes any access_key or secret_key arguments. I removed the hard-coded AWS credentials that an earlier version of this file had, and now rely on the AWS CLI's configured default profile instead — keeping terraform.tfvars and any file containing secrets out of version control.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "6.41.0"
    }
  }

 backend "s3" {
    bucket       = "dway-terraform-state-bucket"
    key          = "terraform-aws-web-server/terraform.tfstate"
    region       = "us-east-2"
    use_lockfile = true
    encrypt      = true
  }
}

# Configure the AWS Provider
provider "aws" {
  region = "us-east-2"
}

# Create a VPC
resource "aws_vpc" "main" {
  cidr_block = var.vpc_cidr_block

  tags = {
    "Name" = "Production ${var.main_vpc_name}"
  }
}
```

main.tf — provider configuration, S3 backend, and VPC resource

Networking: Subnet, Internet Gateway, and Routing

With the VPC in place, I created an aws_subnet resource called “web” inside it, using a CIDR block and availability zone that are both pulled from variables. This subnet is where the EC2 instance will live.

Next, I attached an aws_internet_gateway to the VPC so that resources inside it can reach the public internet. Finally, I used an aws_default_route_table resource to modify the VPC's default route table directly, adding a route that sends all outbound traffic (0.0.0.0/0) through the internet gateway. Using the default route table here, rather than creating a brand-new one, keeps the configuration simpler since every subnet in this VPC already uses that default table.

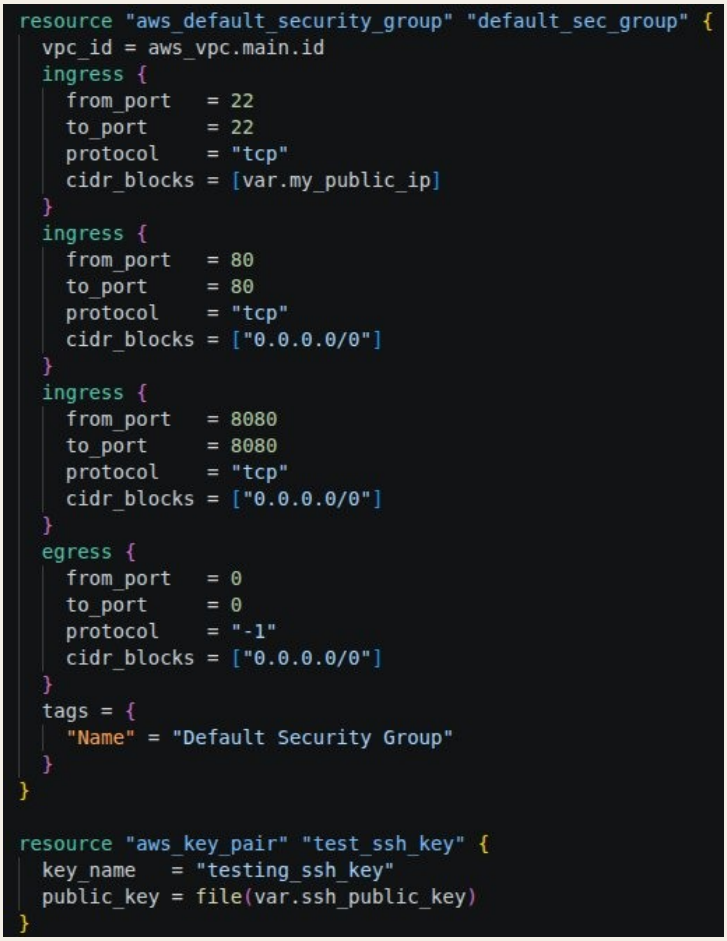
```
# Create a subnet
resource "aws_subnet" "web" {
  vpc_id = aws_vpc.main.id
  cidr_block = var.web_subnet
  availability_zone = var.subnet_zone
  tags = {
    "Name" = "Web subnet"
  }
}

resource "aws_internet_gateway" "my_web_igw" {
  vpc_id = aws_vpc.main.id
  tags = {
    "Name" = "${var.main_vpc_name} IGW"
  }
}

resource "aws_default_route_table" "main_vpc_default_rt" {
  default_route_table_id = aws_vpc.main.default_route_table_id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.my_web_igw.id
  }
  tags = {
    "Name" = "my-default-rt"
  }
}
```

main.tf — subnet, internet gateway, and default route table

Security Group

Security groups act as a virtual firewall for an instance, controlling which traffic is allowed in and out. Instead of creating a brand-new security group, I customized the aws_default_security_group resource tied to the VPC, which manages the VPC's built-in default group.

I opened three inbound ports: port 22 for SSH so I can connect to and manage the instance, port 80 for standard HTTP traffic to reach the Apache web server, and port 8080 so the Nginx container started by the entry script is reachable from outside the instance. Egress is left fully open (all ports, all protocols) so the instance can reach out to the internet for package installs and Docker image pulls. SSH access is restricted to my own public IP address via the my_public_ip variable, rather than allowing connections from anywhere (0.0.0.0/0), which keeps the management port closed to everyone except me.



```
resource "aws_default_security_group" "default_sec_group" {
  vpc_id = aws_vpc.main.id
  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.my_public_ip]
  }
  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    from_port   = 8080
    to_port     = 8080
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
  tags = {
    "Name" = "Default Security Group"
  }
}

resource "aws_key_pair" "test_ssh_key" {
  key_name   = "testing_ssh_key"
  public_key = file(var.ssh_public_key)
}
```

main.tf — aws_default_security_group restricted to a known IP, and aws_key_pair resource

SSH Key Pair and AMI Lookup

To allow secure SSH access to the instance, I created an aws_key_pair resource named “test_ssh_key”. Rather than typing the public key directly into the configuration, I used Terraform's file() function to read it from the path stored in the ssh_public_key variable, which points to a key generated locally on my Ubuntu machine.

Rather than hard-coding an AMI ID, which can change between regions and over time, I used a data “aws_ami” block to look it up dynamically. The data source filters AMIs owned by Amazon, sets most_recent to true, and matches on both the AMI name pattern (amzn2-ami-kernel-*-x86_64-gp2) and the x86_64 architecture. This guarantees the configuration always picks up the latest Amazon Linux 2 AMI available in us-east-2 at apply time.

```
resource "aws_key_pair" "test_ssh_key" {
  key_name = "testing_ssh_key"
  public_key = file(var.ssh_public_key)
}

data "aws_ami" "latest_amazon_linux2" {
  owners = ["amazon"]

  most_recent = true
  filter {
    name = "name"
    values = ["amzn2-ami-kernel-*-x86_64-gp2"]
  }

  filter {
    name = "architecture"
    values = ["x86_64"]
  }
}
```

main.tf — aws_key_pair resource and aws_ami data source

EC2 Instance and Provisioners

The aws_instance resource named “my_vm” ties everything together. It uses the AMI ID returned by the data source, launches as a t2.micro, places the instance in the web subnet, attaches the default security group, requests a public IP address, and references the key pair created earlier.

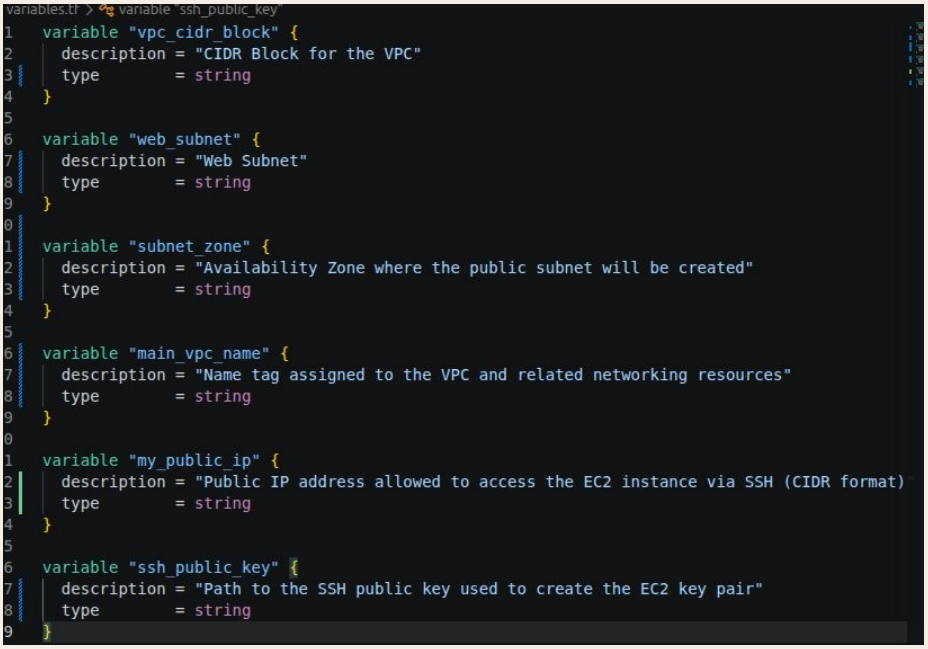
A connection block tells Terraform how to reach the instance over SSH once it's running, using the ec2-user account and a private key stored locally. Two provisioners then run automatically as part of terraform apply: a file provisioner uploads entry-script.sh to the instance's home directory, and a remote-exec provisioner makes the script executable and runs it with sudo. This means the instance is fully configured — web server and container included — by the time terraform apply finishes, with no manual SSH session required afterward.

```
resource "aws_instance" "my_vm" {
  ami                         = data.aws_ami.latest_amazon_linux2.id
  instance_type               = "t2.micro"
  subnet_id                   = aws_subnet.web.id
  vpc_security_group_ids      = [aws_default_security_group.default_sec_group.id]
  associate_public_ip_address = true
  key_name                    = aws_key_pair.test_ssh_key.key_name
  # user_data = file ("entry-script.sh")

  connection {
    type        = "ssh"
    host        = self.public_ip
    user        = "ec2-user"
    private_key = file("~/.ssh/terraform_ec2_key")
  }

  provisioner "file" {
    source      = "./entry-script.sh"
    destination = "/home/ec2-user/entry-script.sh"
  }

  provisioner "remote-exec" {
    inline = [
      "chmod +x /home/ec2-user/entry-script.sh",
      "sudo /home/ec2-user/entry-script.sh",
      "exit"
    ]
  }
  tags = {
    "Name" = "My EC2 Instance - Amazon Linux 2"
  }
}
```

main.tf — aws_instance resource, connection block, and provisioners

Input Variables

variables.tf declares every variable referenced in main.tf, and every one of them now includes both a description and an explicit type. vpc_cidr_block, web_subnet, subnet_zone, main_vpc_name, my_public_ip, and ssh_public_key are all declared as strings with a clear description of what each value represents, including my_public_ip noting that it should be supplied in CIDR format for use in the security group's SSH rule. Documenting every variable this way lets Terraform validate input before it ever reaches AWS, and makes the configuration easy for someone else (or future me) to understand at a glance, instead of needing to cross-reference terraform.tfvars to figure out what a bare variable block is for.



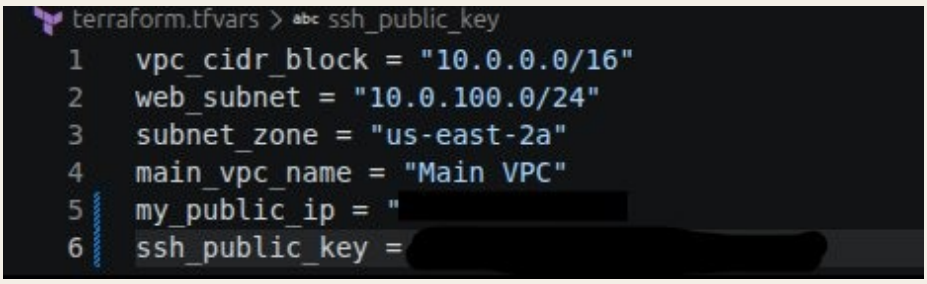
variables.tf — every variable with a description and explicit type

Variable Values

terraform.tfvars supplies the concrete values for every variable declared in variables.tf. Keeping these values in a separate file from the resource definitions in main.tf means the same configuration could be reused for a different environment just by swapping in a different tfvars file, without touching the core infrastructure code.

This file sets the VPC CIDR block to 10.0.0.0/16, the web subnet to 10.0.100.0/24 inside the us-east-2a availability zone, names the VPC “Main VPC,” supplies my own public IP for the security group's SSH restriction, and points to the local path of the SSH public key file used to create the key pair in AWS.

This file contains a real local file path and a real public IP address, so I redacted both values in the screenshot below. Treating terraform.tfvars as semi-sensitive is good practice in general: adding a .gitignore entry for terraform.tfvars (or for any file matching *.tfvars) keeps these kinds of details out of a public repository.

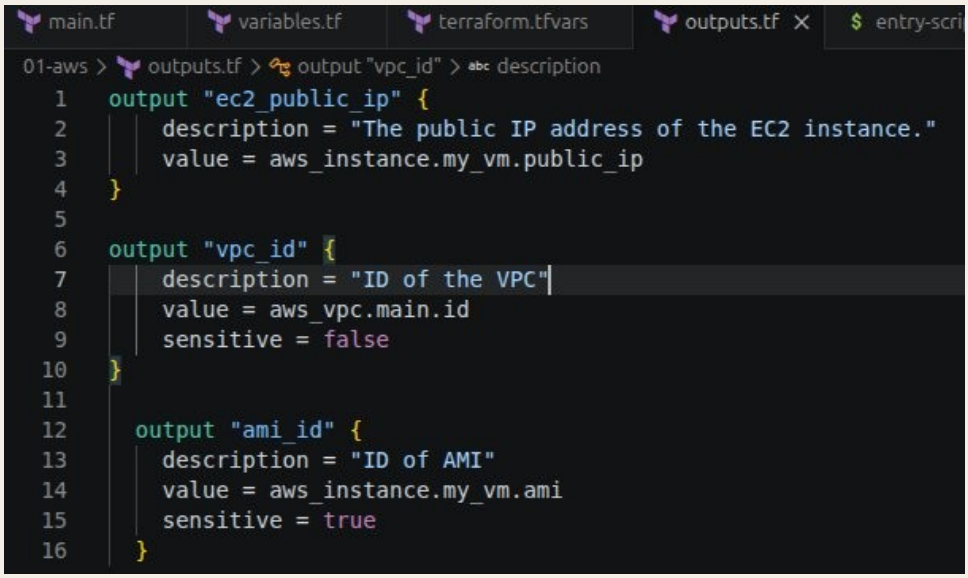


terraform.tfvars — variable values for this environment

Outputs

outputs.tf defines three values that Terraform prints to the terminal after a successful apply: ec2_public_ip, the address I use to actually reach the web server and container in a browser; vpc_id, the identifier of the VPC that was created; and ami_id, the AMI used to launch the instance.

I marked ami_id as sensitive, which tells Terraform to hide its value in CLI output and logs by default, even though an AMI ID isn't truly secret information. vpc_id is explicitly marked sensitive = false, which is the default behavior but makes the intent clear to anyone reading the file. These outputs make it possible to immediately grab the instance's public IP, for example, without having to look it up manually in the AWS console.



outputs.tf — output values returned after terraform apply

Bootstrap Script: entry-script.sh

entry-script.sh is the shell script that the file and remote-exec provisioners deliver to the EC2 instance and run automatically during terraform apply. It performs four jobs in sequence: it updates the system's packages and installs the Apache web server (httpd); it starts and enables httpd so the web server survives a reboot, then writes a simple “Deployed via Terraform” heading to the default web root; it installs Docker (trying amazon-linux-extras first, falling back to yum), starts and enables the Docker service, and adds the ec2-user account to the docker group; and finally it pulls and runs the official Nginx image as a background container, mapping container port 80 to host port 8080.

Put together with the security group rules from main.tf, this means that as soon as the apply finishes, the instance's public IP serves the Apache test page on port 80 and an independent Nginx welcome page on port 8080, all without a single manual SSH command.

entry-script.sh — instance bootstrap script

Lessons Learned and Next Steps

Building this project end-to-end surfaced a few things I changed along the way to move this configuration closer to production-ready, plus a couple of items I'd still tackle next:

- Removed AWS credentials from main.tf entirely. Hard-coded access and secret keys never belong in a Terraform file, even a private one, so the provider block now relies on the AWS CLI's configured default profile instead.
- Restricted SSH (port 22) to my own IP address using the my_public_ip variable, instead of leaving it open to 0.0.0.0/0.
- Added descriptions and explicit types to every variable in variables.tf for clarity and built-in validation.
- Configured Terraform to use an Amazon S3 backend for remote state storage with encryption and state locking, to support collaborative infrastructure management and reduce the risk of state conflicts.
- Still on my list: re-enabling the user_data argument, or relying on it instead of the file/remote-exec provisioners, since user_data runs at boot without requiring an open SSH connection during apply, which is generally considered the more reliable bootstrap method for EC2.

Project Reflection

This project took roughly an hour from writing the first resource block to a fully running web server and container reachable over the public internet. The most challenging part was getting the provisioner connection block right, since it depends on the SSH key, the security group, and the public IP all being correctly wired together before Terraform can connect. The most useful part was seeing how cleanly variables and tfvars separate configuration from code, which makes the same main.tf reusable across different environments just by swapping values.